

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

Кафедра програмних засобів

ЗВІТ

Дисципліна «Конструювання програмного забезпечення»

Робота №7

Тема «Класи-колекції»

Виконав варіант 19

Студент КНТ-122

Онищенко О.А.

Прийняли

Викладач

Леощенко С.Д.

2025

МЕТА

Навчитися основним принципам роботи із класами-колекціями в Java.

ЗАВДАННЯ

1. Ознайомитися з основними теоретичними відомостями за темою роботи, використовуючи ці методичні вказівки, а також рекомендовану літературу.

2. Вивчити основні принципи роботи Java з рядками.

3. Виконати наступні завдання:

7) Phone: id, Прізвище, Ім'я, По батькові, Адреса, Номер кредитної картки, Дебет, Кредит, Час міських і міжміських розмов. Створити масив об'єктів.

Вивести:

а) відомості про абонентів, у яких час внутріміських розмов перевищує задане;

б) відомості про абонентів, які користувалися міжміським зв'язком; в) відомості про абонентів за абеткою.

Задати дві стеки, поміняти інформацію місцями.

3. Оформити звіт з роботи.

4. Відповісти на контрольні питання.

ТЕОРІЯ

Для взаємодії з файлами можна оголосити об'єкт типу File та ініціалізувати його з назвою файлу. Прочитати його вміст можна об'єктом типу Scanner. В нього є метод hasNextLine, який показує чи є у файлі наступні рядки. Для отримання рядку з файлу можна використати метод nextLine.

ВИКОНАННЯ

```
import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Scanner;
import java.util.Stack;

public class Phone {
    public static void main(String[] args) {
        ArrayList<Map<String, String>> phones = new ArrayList<Map<String,
String>>();
        try {
            File file = new File("phones.txt");
            Scanner scanner = new Scanner(file);
            while (scanner.hasNextLine()) {
                String[] values = scanner.nextLine().split(",");
                Map<String, String> phone = new LinkedHashMap<>();
                phone.put("id", values[0]);
                phone.put("surname", values[1]);
                phone.put("name", values[2]);
                phone.put("middleName", values[3]);
                phone.put("address", values[4]);
                phone.put("cardNumber", values[5]);
                phone.put("debit", values[6]);
                phone.put("credit", values[7]);
                phone.put("cityTime", values[8]);
                phone.put("interTime", values[9]);
                phones.add(phone);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }

        Integer cityTime = 50;
        System.out.println("\nCustomers with inner city time of more than "
+ cityTime + ": ");
        for (Map<String, String> map : phones) {
            if (Integer.parseInt(map.get("cityTime")) <= cityTime)
                continue;
        }
    }
}
```

```

        System.out.println("- " + map.get("name") + " from " +
map.get("address") + " with credit card number of "
        + map.get("cardNumber") + " has inner city limit of " +
map.get("cityTime"));
    }

    System.out.println("\nCustomers who used intercity time:");
    for (Map<String, String> map : phones) {
        if (Integer.parseInt(map.get("interTime")) <= 0)
            continue;
        System.out.println(
            "- " + map.get("surname") + " " + map.get("name") + " used "
+ map.get("interTime") + " intercity minutes");
    }

    System.out.println("\nAlphabetical customers:");
    for (char c = 'A'; c <= 'Z'; c++) {
        System.out.print(c + " ");
        for (Map<String, String> map : phones) {
            if (!map.get("name").startsWith(String.valueOf(c)))
                continue;
            System.out.print(map.get("name"));
        }
        System.out.println();
    }

    // Stacks
    System.out.println("\nStacks");

    Stack<Integer> one = new Stack<>();
    Stack<Integer> two = new Stack<>();
    for (int i = 0; i < 5; i++) {
        one.push(i);
    }
    for (int j = 4; j >= 0; j--) {
        two.push(j);
    }
    System.out.println("One: " + one);
    System.out.println("Two: " + two);
    Stack<Integer> oneNew = new Stack<>();
    Stack<Integer> twoNew = new Stack<>();
    for (int i = 0; i < 5; i++) {
        int fromOne = (int) one.pop();
        int fromTwo = (int) two.pop();
        oneNew.push(fromOne);
        twoNew.push(fromTwo);
    }
    System.out.println("One: " + oneNew);
    System.out.println("Two: " + twoNew);
}
}

```

Customers with inner city time of more than 50:

- Anton from Zrazkova 74 with credit card number of 5348 7892 8954 2348 has inner city limit of 120
- Name from Mlynova 32 with credit card number of 9652 5849 1298 5478 has inner city limit of 60
- Other from Berezhna 71 with credit card number of 1203 6584 2340 6540 has inner city limit of 70

Customers who used intercity time:

- Barabash Anton used 10 intercity minutes
- Bugar Maksym used 60 intercity minutes
- Some Other used 10 intercity minutes

Alphabetical customers:

A Anton
B
C
D
E
F
G
H
I
J
K
L
M Maksym
N Name
O Other
P
Q
R
S
T
U
V Vladyslav
W
X
Y
Z

Stacks

One: [0, 1, 2, 3, 4]

Two: [4, 3, 2, 1, 0]

One: [4, 3, 2, 1, 0]

Two: [0, 1, 2, 3, 4]

Рисунок 1.1 — Результати

ВИСНОВКИ

Після виконання роботи було навчено основним принципам роботи із класами-колекціями в Java.

ПИТАННЯ

Основне призначення класу Vector у мові Java?

Створення масивів змінного розміру. Клас Vector реалізує інтерфейс List.

Які основні методи і яке їх застосування, класу Vector ви знаєте?

Основні методи:

- addElement: додає елемент у кінець;
- insertElementAt: додає елемент за індексом;
- removeElementAt: видаляє елемент за індексом.

Основне призначення класу Stack у мові Java?

Створення структури даних стеку, до якої можна додавати і видаляти елементи лише з кінця.

Які основні методи і яке їх застосування, класу Stack ви знаєте?

Основні методи:

- push: додає елемент в кінець;
- pop: видаляє елемент з кінця;
- isEmpty: показує чи стек порожній.

Основне призначення класу Hashtable у мові Java?

Створення словників типу ключ: значення.